SOFTWARE REQUIREMENTS SPECIFICATION

IRO – Issues Reporting Outlet

A web-based system for relating and reporting issues concerning AAUA students and indigenes of Akungba, Iwaro, Etioro, Ayegunle and neighbouring communities

Course: CSC426 – Weekly Dev, Weekend 3

Lecturer: Dr. O. O. Ajayi

Prepared by: Opeyemi Oyinlola

Matric No.: _____

Department: Computer Science, Adekunle Ajasin University, Akungba-Akoko

Version: 1.0

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture and Component Interactions
4. Functional Requirements
5. Non-Functional Requirements
6. Use-Case Descriptions
7. Data Requirements
8. Future Enhancements

1. Introduction

1.1 Purpose

This document specifies the requirements for IRO (Issues Reporting Outlet), a web-based system through which AAUA students and indigenes of the host communities can report, relate and follow up issues that affect them. It describes the system's major components and how they interact, together with its functional and non-functional requirements. It is intended for the client firm, the developer, the lecturer/assessor and future maintainers.

1.2 Scope

IRO provides a single, trusted channel for town-and-gown issues such as insecurity, rent and accommodation disputes, student–indigene conflicts, harassment and extortion, land disputes, roads and transport, water and electricity, sanitation and health. Reporters can submit issues (optionally anonymously), receive a tracking ID and follow progress. An administrator (the IRO Desk – a joint university and community liaison office) reviews, prioritises, responds to and resolves issues.

Because the issues are sensitive, the system emphasises anonymity, confidentiality of reporter details and clear accountability through status updates. The current release is a lightweight prototype built with HTML, CSS and JavaScript, storing data in the browser (localStorage).

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
IRO	Issues Reporting Outlet – the system specified in this document
AAUA	Adekunle Ajasin University, Akungba-Akoko
Indigene	A native or resident of Akungba, Iwaro, Etioro, Ayegunle or neighbouring communities
Reporter	Any user (student or indigene) who submits an issue
IRO Desk / Admin	Authorised officer(s) who manage reported issues
Tracking ID	Unique code (IRO-XXXXXX) issued for every report
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
localStorage	Browser storage used to persist data in the prototype

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- CSC426 Weekly Dev – Weekend 3 brief, Dr. O. O. Ajayi (01/07/26).
- IRO community questionnaire (Google Form) and Ishikawa analysis (Section 2.6).

1.5 Overview

Section 2 gives an overall description of the product, its users and constraints, including the problem analysis. Section 3 describes the system architecture and how the major components interact. Section 4 lists functional requirements, Section 5 non-functional requirements, Section 6 the use cases and Section 7 the data requirements.

2. Overall Description

2.1 Product Perspective

IRO is a new, self-contained web application. It replaces informal channels (word of mouth, social media posts, letters to student union or community leaders) that are slow, untracked and unsafe for reporters. It runs in any modern web browser and needs no installation.

2.2 Product Functions

- Submit an issue as a Student or an Indigene, optionally anonymously.
- Classify the issue by category, community and priority.
- Generate a tracking ID and let the reporter track progress.
- Let the IRO Desk log in, view statistics, filter and search issues.
- Let the IRO Desk update status, respond to reporters and delete invalid reports.

2.3 User Classes and Characteristics

User class	Description	Technical skill
Student	Registered AAUA student living on or off campus in host communities	Basic smartphone/web use
Indigene / Resident	Natives, landlords, traders and residents of Akungba, Iwaro, Etioro, Ayegunle etc.	Varies; low to moderate
Admin (IRO Desk)	University and community liaison officers who manage issues	Moderate

2.4 Operating Environment

- Client: any modern browser (Chrome, Firefox, Edge, Safari) on desktop or mobile.
- Hosting: static web hosting such as GitHub Pages; no server runtime needed for the prototype.

2.5 Design and Implementation Constraints

- Built with HTML5, CSS3 and vanilla JavaScript to limit complexity within the submission timeframe.
- Prototype data is stored in the browser (localStorage), so reports are visible only on the device where they were made. A production version would use a central database.
- The demo admin account (admin / iro2026) is for demonstration only.
- Source code and documentation must be stored in a GitHub repository.

2.6 Problem Analysis (Ishikawa)

An Ishikawa (fishbone) analysis was carried out to identify the root causes of the problem "issues affecting AAUA students and indigenes go unreported, unheard or unresolved". Causes were grouped under People, Methods/Process, Communication, Technology, Environment and Management/Policy. IRO addresses the causes that software can solve: a single formal channel, anonymous reporting, a central issue database, tracking and feedback to reporters, and a joint IRO Desk with clear accountability.

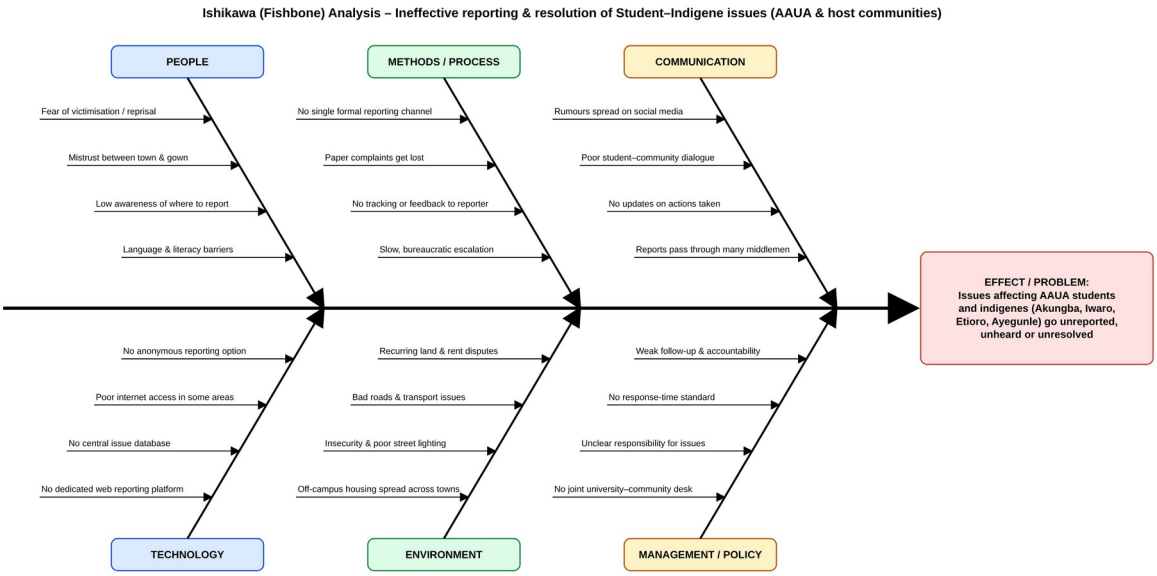


Figure 1: Ishikawa diagram of the IRO problem domain (source: docs/diagrams/ishikawa_iro.drawio)

2.7 Assumptions and Dependencies

- Users have access to an internet-enabled phone or computer.
- The university and community leaders will jointly staff the IRO Desk.
- Emergencies (e.g. violent attacks) are still reported to the police; IRO complements but does not replace emergency services.

3. System Architecture and Component Interactions

3.1 Major Components

Component	File(s)	Responsibility
Presentation layer (UI)	index.html, report.html, track.html, admin.html, assets/style.css	Pages, forms, tables and responsive layout shown to users
Report module	app.js – initReportPage(), validateReport()	Collects, validates and submits issue reports; shows tracking ID
Tracking module	app.js – initTrackPage()	Looks up an issue by tracking ID and shows its status timeline
Admin module	app.js – initAdminPage(), renderDashboard()	Login, statistics, filtering, status updates and deletion
Issue model	app.js – class Issue	Represents one issue; generates ID; records status updates
Data access layer	app.js – class IssueStore	Loads, saves, finds, updates and deletes issues; computes statistics
Authentication	app.js – class Auth	Validates admin credentials and manages the session
Storage	Browser localStorage / sessionStorage	Persists issues (localStorage) and admin session (sessionStorage)

3.2 Component Interactions

3.2.1 Submitting an issue

- The reporter fills the form on report.html (Presentation layer).

- The Report module calls validateReport(); errors are shown and nothing is saved if validation fails.
- On success, a new Issue object is created; its constructor generates the tracking ID and an initial "Pending" update. If anonymous, name and phone are blanked.
- IssueStore.add() saves the issue to Storage; the UI displays the tracking ID with a link to track it.

3.2.2 Tracking an issue

- The reporter enters a tracking ID on track.html (or follows the link).
- The Tracking module calls IssueStore.find(id), which reads Storage and returns the Issue.
- The UI renders status and update timeline only – no personal details are shown.

3.2.3 Managing issues (Admin)

- The admin logs in on admin.html; Auth.login() validates credentials and sets a session flag.
- renderDashboard() requests IssueStore.stats() and IssueStore.load(), applies filters and draws the table.
- On Update, Issue.addUpdate(status, note) appends to the history and IssueStore.update() saves it; the reporter sees the note when tracking.
- On Delete (after confirmation), IssueStore.remove(id) removes the issue. Logout calls Auth.logout().

3.3 Use-Case Diagram

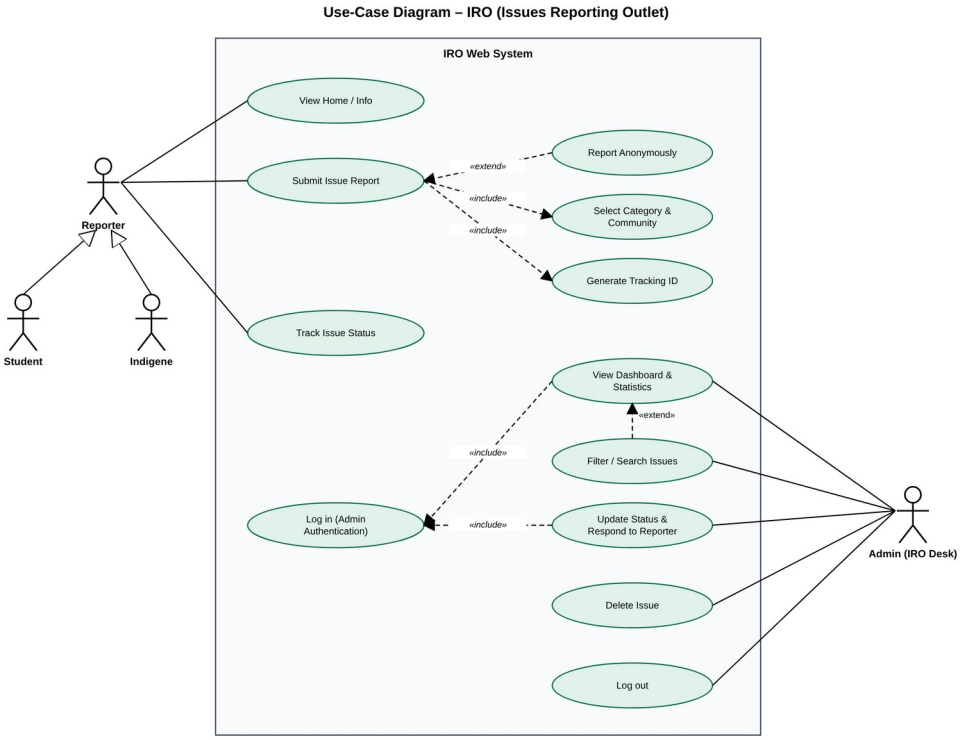


Figure 2: Use-case diagram of IRO (source: docs/diagrams/usecase_iro.drawio)

4. Functional Requirements

ID	Requirement	Description	Priority
FR-01	Reporter type	The system shall require every reporter to identify as either an AAUA Student or an Indigene/Resident.	High
FR-02	Submit issue	The system shall allow a reporter to submit an issue with category, community, priority, title, specific location and description.	High

ID	Requirement	Description	Priority
FR-03	Anonymous reporting	The system shall allow a reporter to submit an issue anonymously; name and phone shall then not be stored.	High
FR-04	Student details	The system shall show an optional Matric Number field only when the reporter is a Student.	Medium
FR-05	Community selection	The system shall provide the host communities Akungba, Iwaro, Etioro, Ayegunle and Other.	High
FR-06	Input validation	The system shall validate required fields, minimum title (5) and description (20) lengths and Nigerian phone format before saving.	High
FR-07	Tracking ID	The system shall generate a unique tracking ID (format IRO-XXXXXX) for each submitted issue and display it to the reporter.	High
FR-08	Track issue	The system shall allow anyone with a tracking ID to view the issue's status and its full update history.	High
FR-09	Admin login	The system shall restrict the dashboard to authenticated administrators (IRO Desk) via username and password.	High
FR-10	Dashboard statistics	The system shall display totals of issues by status and by reporter type (students vs indigenes).	Medium
FR-11	Filter & search	The system shall let the admin filter issues by status, reporter type and category, and search by title, description or ID.	Medium
FR-12	Update status	The system shall let the admin change an issue's status (Pending, Under Review, Resolved, Rejected) and attach a response note visible to the reporter.	High
FR-13	Delete issue	The system shall let the admin delete an issue after confirmation.	Low
FR-14	Logout	The system shall let the admin end the session; dashboard pages shall then be hidden.	Medium
FR-15	Public summary	The home page shall show counts of issues reported, resolved and in progress.	Low

5. Non-Functional Requirements

ID	Attribute	Requirement
NFR-01	Security & Privacy	Anonymous reports shall store no personal identifiers. All user input shall be HTML-escaped before display to prevent script injection. Admin functions require login.
NFR-02	Confidentiality	Reporter name, phone and matric number shall be visible only on the admin dashboard, never on the public tracking page.
NFR-03	Usability	A first-time user shall be able to submit an issue in under 3 minutes with no training. Plain English labels and hints shall be used.
NFR-04	Performance	Each page shall load and respond to user actions in under 2 seconds on a 3G connection.
NFR-05	Availability	The system shall be available 24/7 when hosted (e.g. GitHub Pages), with no server dependency for the prototype.
NFR-06	Portability / Compatibility	The system shall work on current Chrome, Firefox, Edge and Safari, on desktop and mobile.
NFR-07	Responsiveness	The layout shall adapt to screens from 360px (phones) upward

ID	Attribute	Requirement
		without horizontal scrolling.
NFR-08	Reliability	Submitted issues shall persist in browser storage across page reloads; invalid input shall never be saved.
NFR-09	Maintainability	Code shall be modular (Issue, IssueStore, Auth classes and per-page init functions), commented and version-controlled on GitHub.
NFR-10	Scalability	The data layer (IssueStore) shall be isolated so it can be replaced by a server database (e.g. MySQL/Firebase) without changing the UI.

6. Use-Case Descriptions

Field	Detail
Use case	UC-01 Submit Issue Report
Actor	Reporter (Student / Indigene)
Pre-condition	Reporter is on report.html
Main flow	1. Reporter selects Student or Indigene. 2. Reporter optionally ticks Report anonymously. 3. Reporter selects community, category and priority, and enters title, location and description. 4. Reporter clicks Submit Issue. 5. System validates input, saves the issue and shows the tracking ID.
Alternative flow	3a. Invalid input: system lists errors; reporter corrects them.
Post-condition	Issue stored with status Pending.

Field	Detail
Use case	UC-02 Track Issue Status
Actor	Reporter
Pre-condition	Reporter has a tracking ID
Main flow	1. Reporter opens track.html and enters the ID. 2. System finds the issue. 3. System shows status and update history.
Alternative flow	2a. ID not found: system shows "No issue found".
Post-condition	Reporter knows current status.

Field	Detail
Use case	UC-03 Update Status & Respond
Actor	Admin (IRO Desk)
Pre-condition	Admin is logged in
Main flow	1. Admin finds the issue (optionally filtering/searching). 2. Admin selects a new status and clicks Update. 3. Admin enters a response note. 4. System records the update and refreshes the dashboard.
Alternative flow	3a. Admin cancels: no change is made.
Post-condition	New status and note visible to the reporter.

Field	Detail
Use case	UC-04 Log in
Actor	Admin (IRO Desk)
Pre-condition	Admin is on admin.html
Main flow	1. Admin enters username and password. 2. System validates credentials. 3. System shows the dashboard.
Alternative flow	2a. Invalid credentials: error shown, dashboard stays hidden.
Post-condition	Admin session started.

7. Data Requirements

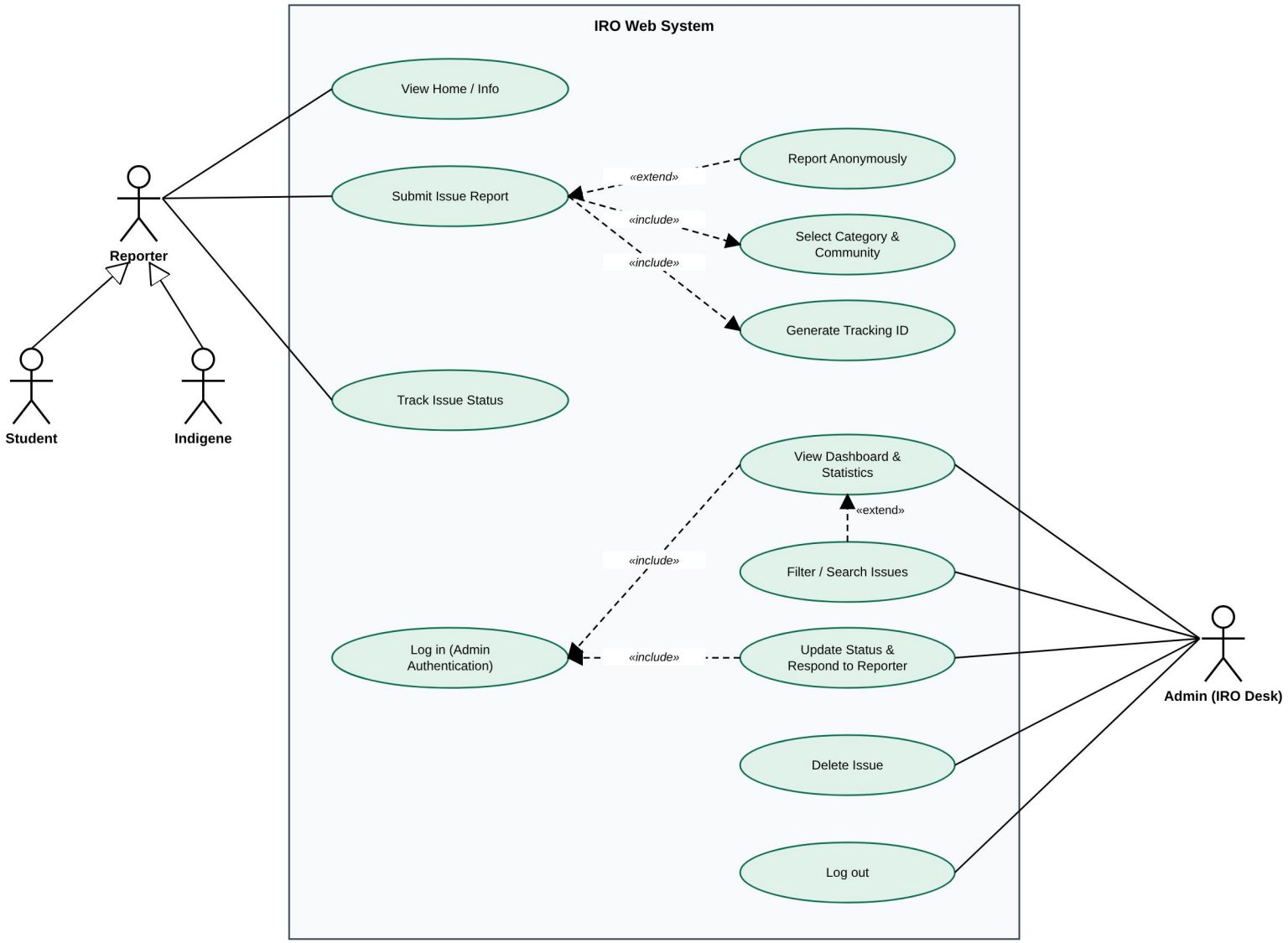
Each issue is stored as a JSON object with the following fields:

Field	Type	Description
id	String	Tracking ID, e.g. IRO-7KQ2MX
reporterType	String	Student or Indigene
anonymous	Boolean	True if the reporter chose anonymity
name, phone	String	Reporter contact (empty when anonymous)
matricNo	String	Optional, students only
community	String	Akungba, Iwaro, Etioro, Ayegunle or Other
category	String	One of 10 issue categories
priority	String	Low, Medium, High or Urgent
title, description, location	String	Details of the issue
status	String	Pending, Under Review, Resolved or Rejected
createdAt	ISO date	Submission date and time
updates	Array	History of {status, note, date} entries

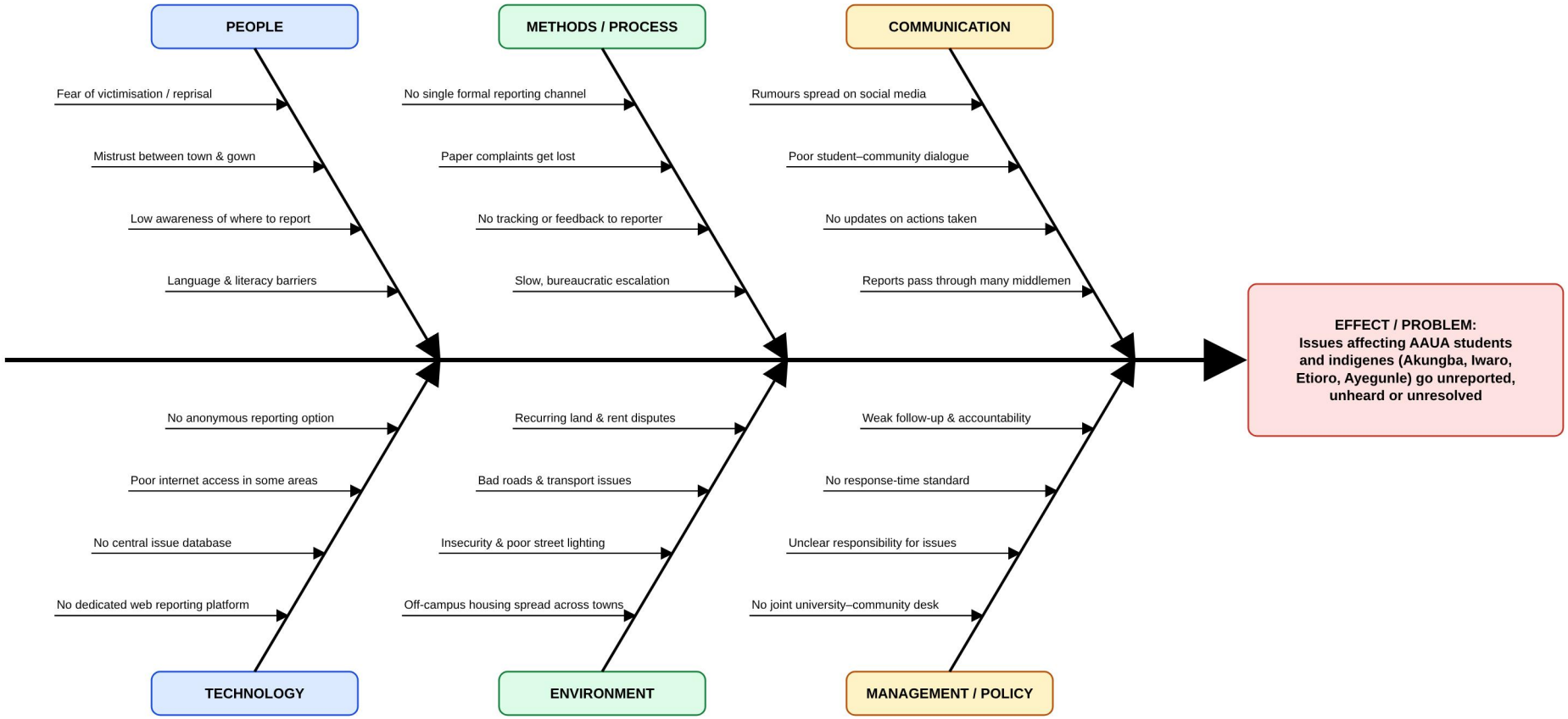
8. Future Enhancements

- Central database and server API (e.g. PHP/MySQL or Firebase) so reports are shared across devices.
- SMS/WhatsApp notifications on status change, and USSD reporting for users without smartphones.
- Yoruba language option, image/evidence upload and role-based accounts for community leaders.

Use-Case Diagram – IRO (Issues Reporting Outlet)



Ishikawa (Fishbone) Analysis – Ineffective reporting & resolution of Student–Indigene issues (AAUA & host communities)



OBJECT LISTING AND CODE LISTING

IRO – Issues Reporting Outlet

CSC426 – Weekly Dev, Weekend 3 | Dr. O. O. Ajayi

Prepared by: Opeyemi Oyinlola | Matric No.: _____

Computer Science, Adekunle Ajasin University, Akungba-Akoko

Part A: Object Listing

A.1 Project files

File	Type	Purpose
app/index.html	HTML page	Home page: introduction, how it works, public statistics
app/report.html	HTML page	Issue reporting form (Student / Indigene, anonymous option)
app/track.html	HTML page	Issue tracking by tracking ID
app/admin.html	HTML page	Admin login and issues dashboard
app/assets/style.css	Stylesheet	Layout, colours, forms, tables, badges, responsive rules
app/assets/app.js	JavaScript	Classes, data storage, validation and page logic

A.2 Class: Issue

Member	Kind	Description
id	Property (String)	Unique tracking ID, e.g. IRO-7KQ2MX
reporterType	Property (String)	"Student" or "Indigene"
anonymous	Property (Boolean)	True when the reporter hides identity
name, phone	Property (String)	Reporter contact; blank/"Anonymous" when anonymous
matricNo	Property (String)	Student matric number (optional)
community	Property (String)	Akungba, Iwaro, Etioro, Ayegunle or Other
category, priority	Property (String)	Issue classification
title, description, location	Property (String)	Issue details
status	Property (String)	Pending, Under Review, Resolved or Rejected
createdAt	Property (ISO date)	Date/time of submission
updates	Property (Array)	History of {status, note, date}
constructor(data)	Method	Builds an Issue from form data or stored JSON
generateId()	Static method	Returns a random IRO-XXXXXX tracking ID
addUpdate(status, note)	Method	Changes status and appends a history entry

A.3 Class: IssueStore (data access)

Member	Kind	Description
load()	Static method	Reads all issues from localStorage and returns Issue objects
saveAll(issues)	Static method	Writes the full list of issues to localStorage
add(issue)	Static method	Adds a new issue to the top of the list
find(id)	Static method	Returns the issue matching a tracking ID
update(issue)	Static method	Replaces a stored issue with its updated version
remove(id)	Static method	Deletes an issue by ID
stats()	Static method	Returns counts by status and reporter type

A.4 Class: Auth (admin session)

Member	Kind	Description
login(username, password)	Static method	Checks credentials; starts session on success
isLoggedIn()	Static method	Returns true if an admin session exists
logout()	Static method	Ends the admin session

A.5 Functions

Function	Used on	Description
$(sel)	All pages	Shortcut for document.querySelector
escapeHTML(str)	All pages	Escapes user text before display (prevents script injection)
formatDate(iso)	Track, Admin	Formats dates for Nigerian locale
badge(status)	Track, Admin	Returns coloured status badge HTML
fillSelect(select, options, placeholder)	Report, Admin	Fills a drop-down list
initHomePage()	index.html	Shows public statistics
initReportPage()	report.html	Sets up form, toggles fields, handles submission
validateReport(form)	report.html	Returns a list of validation errors
initTrackPage()	track.html	Looks up and displays an issue's timeline
initAdminPage()	admin.html	Handles login/logout and filter events
renderDashboard()	admin.html	Draws statistics and the issues table; handles update/delete

A.6 Constants

Constant	Type	Value / Description
STORAGE_KEY	String	"iro_issues" – localStorage key for issues
SESSION_KEY	String	"iro_admin_session" – sessionStorage key
ADMIN	Object	Demo admin credentials {username, password}
CATEGORIES	Array	10 issue categories
COMMUNITIES	Array	Akungba, Iwaro, Etioro, Ayegunle, Other
STATUSES	Array	Pending, Under Review, Resolved, Rejected
PRIORITIES	Array	Low, Medium, High, Urgent

A.7 Main interface objects (HTML elements)

Element ID	Page	Description
#reportForm	report.html	Issue report form
reporterType (radio)	report.html	Student / Indigene choice
#anonymous	report.html	Report-anonymously checkbox
#name, #phone, #matricNo	report.html	Reporter identity fields
#community, #category, #priority	report.html	Classification drop-downs
#title, #location, #description	report.html	Issue detail fields

Element ID	Page	Description
#formMessage	report.html	Success / error message box
#trackForm, #trackId, #trackResult	track.html	Tracking form, input and result area
#loginForm, #username, #password	admin.html	Admin login form
#dashboard, #stats, #issueRows	admin.html	Dashboard, statistics and issues table
#fStatus, #fType, #fCategory, #fSearch	admin.html	Dashboard filters
#homeStats	index.html	Public statistics

Part B: Code Listing (GitHub)

The full source code is hosted on GitHub instead of being printed here.

Repository: https://github.com/opes1/IRO-Issues-Reporting-Outlet

Live app: https://opes1.github.io/IRO-Issues-Reporting-Outlet/

File	GitHub link	Lines
app/assets/app.js	https://github.com/opes1/IRO-Issues-Reporting-Outlet/blob/main/app/assets/app.js	375
app/assets/style.css	https://github.com/opes1/IRO-Issues-Reporting-Outlet/blob/main/app/assets/style.css	126
app/index.html	https://github.com/opes1/IRO-Issues-Reporting-Outlet/blob/main/app/index.html	56
app/report.html	https://github.com/opes1/IRO-Issues-Reporting-Outlet/blob/main/app/report.html	99
app/track.html	https://github.com/opes1/IRO-Issues-Reporting-Outlet/blob/main/app/track.html	40
app/admin.html	https://github.com/opes1/IRO-Issues-Reporting-Outlet/blob/main/app/admin.html	72